

Lab 3: ML Workloads Part III (Rooflines + NVIDIA-SMI)

CS2470 Advanced Computer Architecture, Fall 2026

This lab asks two questions about running AI models on a GPU. First, what limits how fast a piece of GPU code runs: the math hardware, or the memory feeding it? Second, how much power and energy does running a language model actually use? The first half answers question one with a **roofline model**. The second half answers question two by logging power with `nvidia-smi` while real models generate text.

If you already know what arithmetic intensity, GEMMs, and prefill vs decode are, skip to [Setup](#).

Background (start here if you're new)

What a GPU is doing

A GPU has two resources that matter here:

- **Compute:** thousands of small math units that do multiply-adds. Speed is measured in **FLOP/s** (floating-point operations per second). 1 TFLOP/s = 10^{12} FLOP/s.
- **Memory bandwidth:** how fast data moves between the GPU's main memory (DRAM, the "23 GB" on the spec sheet) and those math units. Measured in **GB/s**.

Every computation needs both: data has to arrive before it can be multiplied. Whichever one runs out first sets the speed. The L4 GPU used in this lab can do 121 TFLOP/s of fp16 math, but its memory only delivers 300 GB/s.

Arithmetic intensity

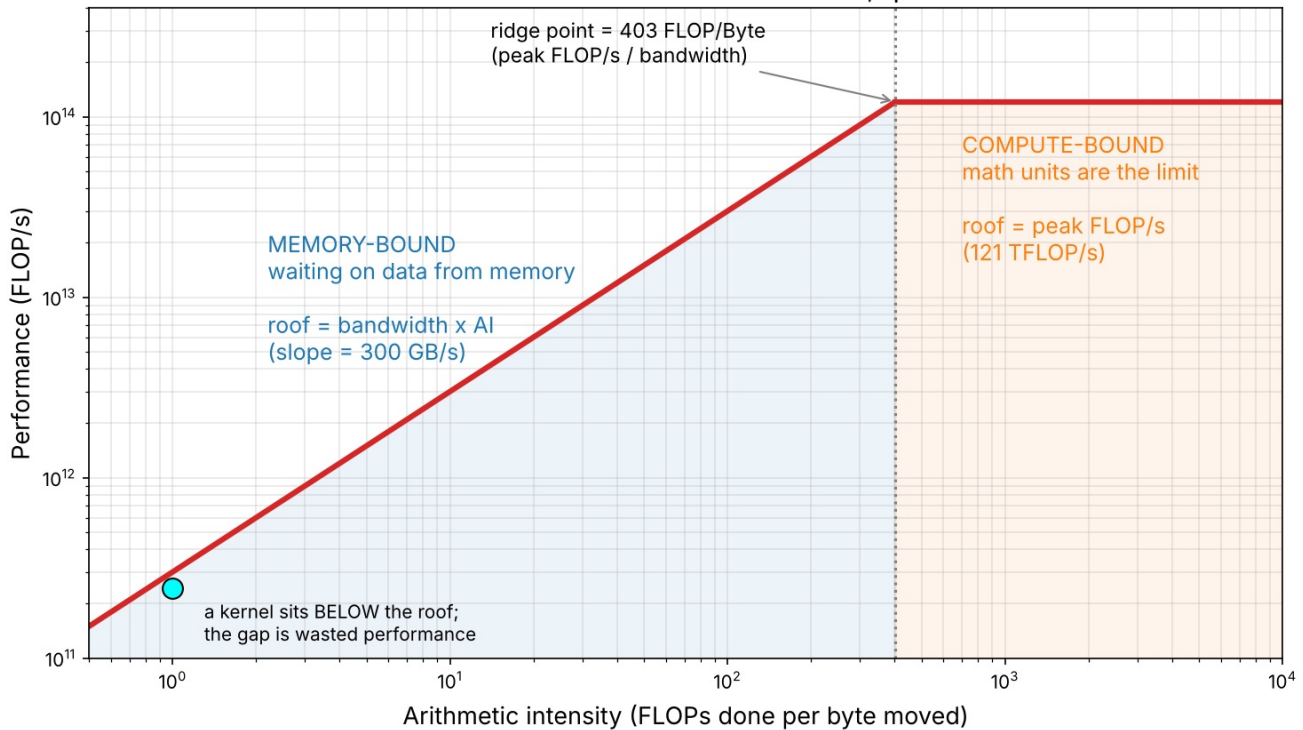
Arithmetic intensity (AI) = FLOPs performed / bytes moved from memory. It measures how much math a program does for each byte it reads.

An analogy: a kitchen with very fast cooks (compute) and a slow delivery truck bringing ingredients (memory). A recipe that does a lot of chopping per ingredient (high AI) keeps the cooks busy. A recipe where each ingredient gets one quick touch (low AI) leaves the cooks waiting for the truck.

The roofline model

A roofline plot puts arithmetic intensity on the x-axis and achieved performance (FLOP/s) on the y-axis. The "roof" is the best performance the hardware could possibly reach at each AI:

How to read a roofline (NVIDIA L4, fp16)



- **Left side (sloped roof):** low AI. Performance is capped at bandwidth $\times$ AI. These programs are **memory-bound**: buying more math units wouldn't help.
- **Right side (flat roof):** high AI. Performance is capped at peak FLOP/s. These programs are **compute-bound**.
- **Ridge point:** where the two meet, at AI = peak FLOP/s / bandwidth. For the L4 that is 121 TFLOP/s / 300 GB/s $\approx$ 403 FLOP/Byte.

To place a real program on the plot, you compute its AI by hand and measure its runtime. The vertical gap between the dot and the roof shows how much performance is being left on the table. The horizontal position tells you which resource to blame.

GEMMs and convolutions

- **GEMM** (general matrix multiply) is $C = A \times B$. Multiplying an $M \times K$ matrix by a $K \times N$ matrix takes $2 \cdot M \cdot N \cdot K$ FLOPs (each output element needs K multiplies and K adds). Almost all the math in a transformer language model is GEMMs: each layer multiplies the token activations by large weight matrices.
- **Convolution** is the core operation of image models (CNNs). It slides a small filter over an image. Under the hood GPUs run it as a GEMM in disguise ("implicit GEMM").

Prefill vs decode (how LLMs generate text)

A language model generates text in two phases:

1. **Prefill:** the model reads the whole prompt at once. With a 1,000-token prompt, every weight matrix is multiplied against 1,000 rows of activations in one go. Each weight loaded from memory gets reused 1,000 times, so AI is high and prefill is **compute-bound**.
2. **Decode:** the model then generates one new token at a time. Each step multiplies every weight matrix against a single row. Each weight is loaded from memory and used once, so AI is about 1 and decode is **memory-bound**. For an 8B-parameter model in fp16, every single token requires streaming ~ 16 GB of weights through the memory system.

Batching means generating for many prompts at once. During decode with batch size 100, each weight read serves 100 rows instead of 1, which raises AI about 100x. That's why batching is the main trick for making LLM serving efficient, and Exercise IV measures the effect directly.

Power vs energy

- **Power** (watts, W) is the rate of energy use at a given moment. `nvidia-smi` reports it.
- **Energy** (joules, J) is the total used over time: $\text{energy} = \text{power} \times \text{time}$. A GPU drawing 70 W for 10 s uses 700 J.
- **TDP / power limit** is the maximum power the GPU is allowed to draw (72 W for the L4, which is small for a data-center GPU, compared with 700 W for an H100). If a workload would exceed it, the GPU lowers its clock speed to stay under, which makes it slower.
- **P-states** are power modes. P0 is full speed, P8 is deep idle with clocks parked.

For energy efficiency, the useful number is usually **energy per unit of work** (here, joules per generated token), not raw power.

How the lab is run

The course cluster uses **Slurm**, a job scheduler. You don't run heavy code on the login machine. Instead you either request an interactive GPU session (`srunch ... --pty bash`) or submit a script to run in the background (`sbunch script.sbatch`). The output of a batch job goes to a log file. **PyTorch Profiler** records how long each GPU operation takes. `nvidia-smi` is NVIDIA's command-line tool for reading GPU status: power, temperature, memory, running processes.

Setup

- **Hardware:** NVIDIA L4 (Ada, 23,034 MiB, 72 W power limit) on nodes `gpu-dy-gpu-cr-1` and `gpu-dy-gpu-cr-7` of the HUIT academic cluster
- **Software:** driver 580.126.20, CUDA 13.0, the `cs2470_env` conda env from Lab 0
- **Workspace:** `~/workspace/ml_workloads_part_3`
- **How I ran it:** batch jobs instead of an interactive `srunch`, because the OOD web shell dropped my interactive session mid-run. `code/run_lab3.sbatch` runs every part in order (job 1700). `code/run_kernels.sbatch` reruns only Part II and the roofline plots (job 1701, see the note in Part II).
- **Precision:** everything is fp16. FLOP counts and byte counts below assume 2 bytes per element.

File	What it does
<code>code/plot_roofline.py</code>	Part I template with the TODOs filled in, plus Exercise I and II plots
<code>code/profile_kernel.py</code>	Part II: times the GEMM and conv kernels with PyTorch Profiler, writes <code>kernels.json</code>
<code>code/profile_power_workload.py</code>	Part IV: logs <code>nvidia-smi</code> in a subprocess while an LLM runs <code>generate()</code>
<code>code/plot_power.py</code>	Exercises III and IV: power over time, idle and TDP lines, energy table
<code>code/run_lab3.sbatch</code>	Runs the whole lab end to end
<code>kernels.json</code>	Measured kernel data used for the roofline plots

To reproduce:

```
cd ~/workspace/ml_workloads_part_3
sbunch run_lab3.sbatch           # ~10 min on an L4, writes lab3_<jobid>.log and results/
```

Part I: Constructing a Roofline

A roofline has two pieces. On the left, performance is capped by memory bandwidth, so attainable FLOP/s = bandwidth × arithmetic intensity (AI). On the right, it is capped by peak compute. The two meet at the ridge point:

```
ridge AI = peak FLOP/s / peak bandwidth (FLOP/Byte)
```

Step 1, hardware data. I used the dense FP16 Tensor Core numbers from NVIDIA's datasheets (without the 2:4 sparsity speedup, since our kernels are dense):

GPU	Peak FP16 (dense)	Memory BW	Ridge point
L4	121 TFLOP/s	300 GB/s	403.3 FLOP/B
A100 SXM 80GB	312 TFLOP/s	2,039 GB/s	153.0 FLOP/B
H100 SXM	989 TFLOP/s	3,350 GB/s	295.2 FLOP/B

Step 2, intersection point. The template's TODO line becomes:

```
x_intersection = peak_flops / peak_memory_bw
intersections.append({'name': name, 'x_intersection': x_intersection, ...})
```

Step 3, plot the curves. A sloped line from `min_x` to the ridge, then a flat line from the ridge to `max_x`:

```
x_mem = np.geomspace(min_x, x_intersection, 200)
ax.plot(x_mem, peak_memory_bw * x_mem, color=color, label=name)           # memory roof
ax.plot([x_intersection, max_x], [peak_flops, peak_flops], color=color) # compute roof
```

The L4's ridge is high (403 FLOP/B) because it has very little bandwidth for its compute. A kernel needs a lot of reuse per byte before the L4 stops being memory-bound.

In plain terms: on an L4, a program has to do about 400 math operations for every byte it reads before the math units become the bottleneck. Anything less, and the GPU spends its time waiting on memory.

Part II: Adding Kernels to the Roofline

A **kernel** is one function that runs on the GPU (one matrix multiply, for example). To put a kernel on the roofline it needs two numbers: its AI (x-axis) and its achieved FLOP/s (y-axis). AI = FLOPs / bytes, and throughput = FLOPs / time. FLOPs and bytes can be worked out on paper from the matrix sizes. Time has to be measured.

For a GEMM (M×K) @ (K×N) :

```
FLOPs = 2·M·N·K          (one multiply + one add per multiply-accumulate)
Bytes = 2·(M·K + K·N + M·N) (read A, read B, write C, once each, fp16)
```

"4096³" means M = K = N = 4096, i.e two 4096×4096 matrices. For this GEMM: 1.374e11 FLOPs, 1.007e8 bytes, so **AI = 1,365 FLOP/B**.

Profiling. I wrapped the GEMM in PyTorch Profiler, ran 10 warmup iterations, then profiled 20 and divided the GPU kernel time by 20. Warmup matters because the first few calls include one-time costs (loading libraries, picking the fastest kernel) that would make the timing look worse than steady state. `torch.cuda.synchronize()` makes the CPU wait for the GPU to finish, since GPU work is launched asynchronously and would otherwise still be running when the timer stops.

```

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA]) as prof:
    for _ in range(20):
        A @ B
        torch.cuda.synchronize()
    # sum only GPU kernel events
    total_us = sum(e.self_device_time_total for e in prof.events() if e.device_type == DeviceType.CUDA)

```

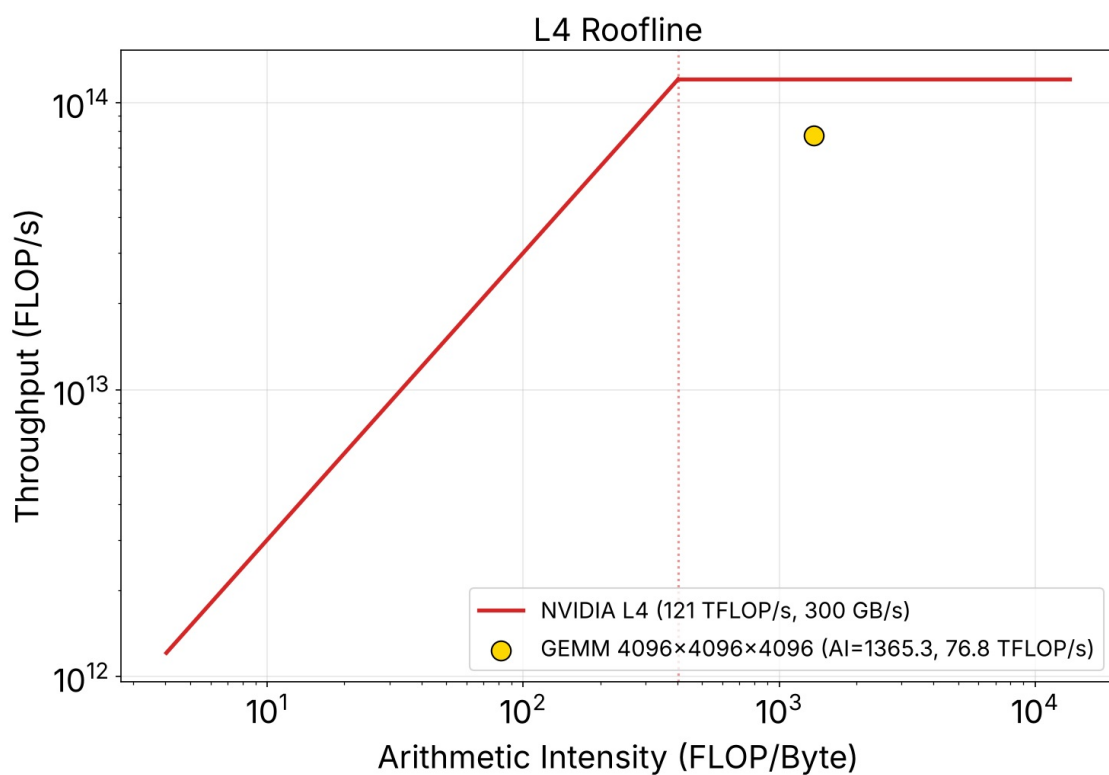
Result: 1.791 ms per call, so **76.8 TFLOP/s**, which is 63% of the L4's 121 TFLOP/s peak. The kernel PyTorch picked is `ampere_fp16_s1688gemm_fp16_128x128_ldg8_f2f_stages_3...` (name truncated in the profiler table).

Note on the timing code: my first version summed `self_device_time_total` across every row of `key_averages()`. That double counts, because CPU ops like `aten::cudnn_convolution` also carry the device time of the kernels they launch. It reported every kernel as ~2x slower than it was. Summing only events whose `device_type` is CUDA matches the profiler's own "Self CUDA time total" line. Job 1701 is the corrected run, and all numbers here come from it.

```

Host: academ-acade-1l73aiWT0xF-c83014867702e61e.alb.us-east-1.amazonaws.com
Themes: Default
===== DONE =====
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
run
Llama-3.2-1B bs=1 time s avg W peak W idle W energy J J/iter J above idle
3.28 61.1 72.7 27.8 200.4 40.1 109.3
energy per generated token: 0.802 J
run
Llama-3.1-8B bs=1 time s avg W peak W idle W energy J J/iter J above idle
15.61 70.1 72.8 27.9 1093.8 218.8 657.5
energy per generated token: 4.375 J
run
Llama-3.1-8B bs=1 time s avg W peak W idle W energy J J/iter J above idle
15.61 70.1 72.8 27.9 1093.8 218.8 657.5
energy per generated token: 4.375 J
GPU 0: NVIDIA L4 (UUID: GPU-9131ad56-ff0b-9343-f69d-09ff85aa0043)
NVIDIA L4:
Kernel time per call: 1790.6 us
GEMM 4096x4096x4096: 1.374e+11 FLOP, 1.007e+08 B, 1.791 ms, AI=1365.33 FLOP/B, 76.75 TFLOP/s
Kernel time per call: 215.5 us
Conv2d 3x3: 7.399e+09 FLOP, 2.576e+07 B, 0.215 ms, AI=287.18 FLOP/B, 34.34 TFLOP/s
Kernel time per call: 11413.9 us
GEMM A 8192x8192x4096: 5.498e+11 FLOP, 2.684e+08 B, 11.414 ms, AI=2048.00 FLOP/B, 48.17 TFLOP/s
Kernel time per call: 274.8 us
GEMM B 1x8192x4096: 6.711e+07 FLOP, 6.713e+07 B, 0.275 ms, AI=1.00 FLOP/B, 0.24 TFLOP/s
NVIDIA L4:
NVIDIA L4:
NVIDIA L4:
GEMM A 8192x8192x4096: AI=2048.00 -> compute-bound, 48.17 TFLOP/s (40% of attainable)
GEMM B 1x8192x4096: AI=1.00 -> memory-bound, 0.24 TFLOP/s (81% of attainable)
(base) [nos673@ip-10-37-33-97 ml_workloads_part_3]$

```



Exercise I: Plotting Rooflines

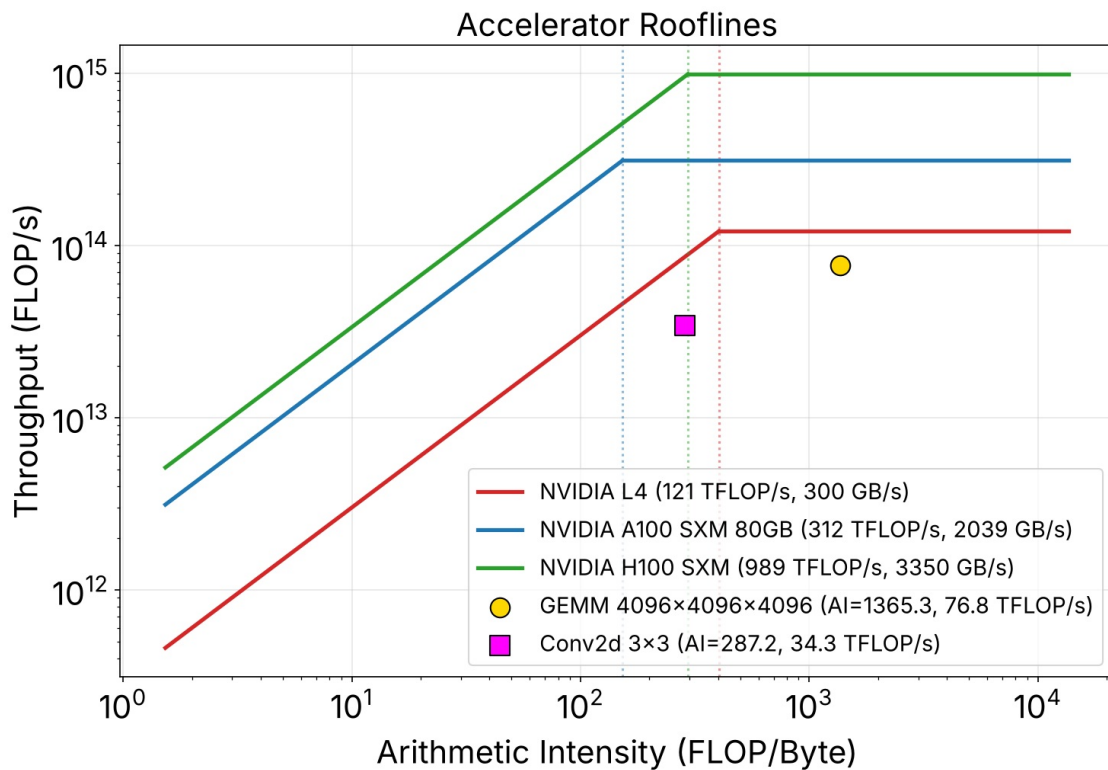
- A100 and H100 rooflines** are added as two more entries in `platforms` (specs in the Part I table).
- Convolution kernel.** I used a ResNet-50 `conv2_x` layer: batch 32, 64→64 channels, 56×56, 3×3, stride 1, padding 1.

```

FLOPs = 2 · N · K · H_out · W_out · C · R · S = 7.399e9
Bytes = 2 · (input + weights + output) = 2.576e7
AI    = 287 FLOP/B

```

Result: 0.215 ms, **34.3 TFLOP/s**.

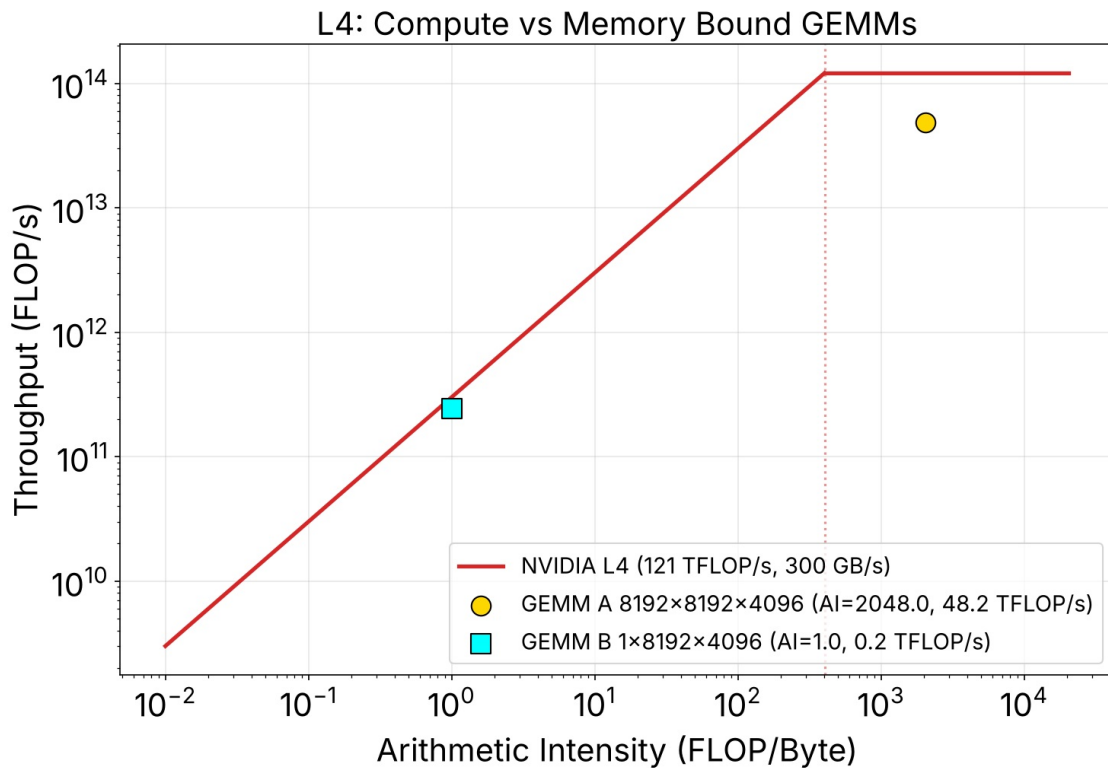


What the plot shows:

- **GEMM (AI 1,365)** is right of every ridge, so it is compute-bound on all three GPUs. On the L4 it reaches 63% of the compute roof.
- **Conv (AI 287)** sits left of the L4 ridge (403) but right of the A100 (153) and H100 (295) ridges. The same kernel is memory-bound on the L4 and compute-bound on the A100 and H100. That's the clearest takeaway of the plot: whether a kernel is "memory-bound" depends on the hardware, not just the kernel.
- **Conv efficiency** is only 40% of its L4 roof (86 TFLOP/s attainable at AI 287). Part of that is layout conversion. The profiler shows the actual convolution kernel (`sm86_xmma_fprop_implicit_gemm_f16f16_f16f32_f32_nhwc...`) is only 52% of the GPU time. The rest is converting the input from PyTorch's default NCHW layout (all of channel 1, then all of channel 2, and so on) to the NHWC layout the kernel wants (all channels of pixel 1, then pixel 2). Same data, different order in memory. Using `channels_last` tensors would remove that.

Exercise II: Compute-Bound vs Memory-Bound Kernels

Kernel	Shape (M, K, N)	AI (FLOP/B)	Time	Throughput	Bound	% of attainable
GEMM A	8192, 8192, 4096	2,048	11.414 ms	48.2 TFLOP/s	compute	40%
GEMM B	1, 8192, 4096	1.0	0.275 ms	0.24 TFLOP/s	memory	81%



In plain terms: GEMM A is a big multiply where every number loaded gets reused thousands of times, so the math units are the bottleneck. GEMM B multiplies a single row by a big matrix, so every number loaded is used once and the GPU mostly waits on memory. Same operation, very different behavior, purely because of the shape.

GEMM A is compute-bound (AI 2,048, about 5x past the ridge). Every weight byte gets reused across 8,192 rows.

GEMM B is memory-bound (AI 1.0, about 400x below the ridge). With $M = 1$, each weight is read once and used for exactly one multiply-add. Almost all the bytes are the 8192×4096 weight matrix (64 MiB). The kernel moves 67 MB in 0.275 ms, which is **244 GB/s, or 81% of the L4's 300 GB/s**. It's doing about as well as the hardware allows, and the only way to speed it up is to move fewer bytes (quantize the weights) or batch more rows.

Mapping to LLM inference:

- **GEMM A = prefill.** The whole prompt (here 8,192 tokens) is processed at once, so M = number of tokens and every weight is reused across all of them.
- **GEMM B = decode.** Each step generates one token, so $M = 1$ and the full weight matrix is streamed from DRAM for a single row of math. This is why decode is memory-bound and why batching (more rows per weight read) helps so much, which Exercise IV confirms.

One more observation: GEMM A only hits 40% of peak while the smaller 4096^3 GEMM hits 63%. I believe this is the 72 W power limit. GEMM A runs 11 ms per call for 20 calls back to back, long enough for the L4 to drop its clocks to stay under the cap (Part IV shows the card pinned at 72 W under any sustained load). I didn't log clocks during this run, so treat this as the likely cause rather than a measured one.

Part III: Basic Measurements with NVIDIA-SMI

```
Host: academ-acade-1L73aiWt8xF-c83014867702e61e.elb.us-east-1.amazonaws.com
NVIDIA-SMI 580.126.20 Driver Version: 580.126.20 CUDA Version: 13.0
+-----+-----+-----+-----+-----+-----+-----+-----+
| GPU | Name | Persistence-M | Bus-Id | Disp.A | Volatile Uncorr. ECC |
| Fan | Temp | Perf | Pwr:Usage/Cap | Memory-Usage | GPU-Util | Compute M. |
|-----+-----+-----+-----+-----+-----+-----+-----+
| 0 | NVIDIA L4 | On | 00000000:31:00.0 Off | 0MiB / 23034MiB | 0% | Default |
| N/A | 36C | P8 | 16W / 72W | | | | N/A |
+-----+-----+-----+-----+-----+-----+-----+-----+
Processes:
+-----+-----+-----+-----+-----+-----+-----+-----+
| GPU | GI | CI | PID | Type | Process name | GPU Memory |
| ID | ID | ID | | | | Usage |
+-----+-----+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+-----+-----+-----+-----+-----+-----+-----+
--- nvidia-smi --lms 1 (2 s sample, header + first lines) ---
timestamp, power.draw [W], temperature.gpu, utilization.gpu [%], memory.used [MiB]
2026/10/09 15:19:52.588, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.594, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.595, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.596, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.597, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.598, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.599, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.601, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.602, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.603, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.604, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.605, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.607, 18.75 W, 36, 0 %, 0 MiB
2026/10/09 15:19:52.608, 18.75 W, 36, 0 %, 0 MiB
===== Part II: kernel profiling =====
(base) [nos673@ip-10-37-33-97 ml_workloads_part_3]$
```

Reading the output for the idle GPU:

Field	Value	Meaning
Power usage	16 W	Current draw. Nothing is running
Power cap	72 W	The power limit (TDP for the L4)
Temperature	36 C	
Perf state	P8	Deepest idle state. Clocks are parked
Memory used	0 MiB / 23,034 MiB	
Processes	none	

`nvidia-smi --lms 1` (and the `--query-gpu ... -lms 1` form used in Part IV) prints a row about every 1 ms. The power value only changes about every 50 ms though, so most rows repeat the previous reading. 1 ms logging doesn't give 1 ms power resolution. In the sample at the bottom of the screenshot, power reads 18.75 W on every row.

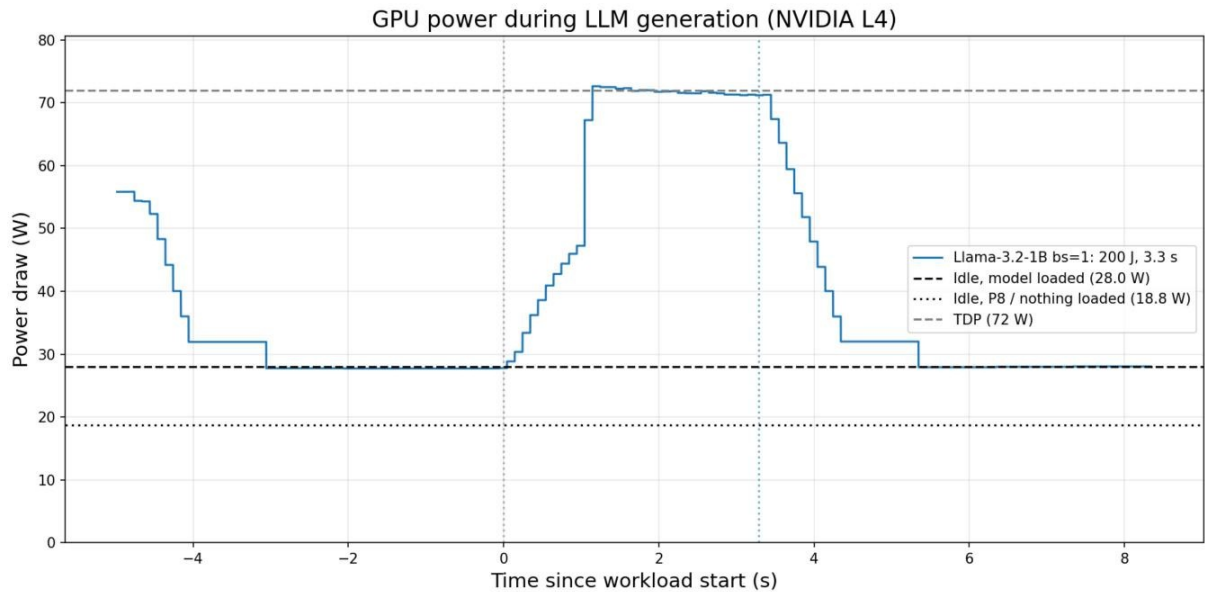
Part IV: Capturing Power During Workload Execution

The idea: start `nvidia-smi` logging power to a file in the background (a **subprocess** is a second program launched from Python that runs alongside it), run the model, then stop the logger and line up the power readings with when the model was running.

`code/profile_power_workload.py` follows the template: launch `nvidia-smi --query-gpu=timestamp,power.draw,... -lms 1 > log.csv` as a subprocess, sleep 5 s, run `5 × generate(max_new_tokens=50)`, sleep 5 s, stop the subprocess. I made three changes:

1. One warmup `generate()` **before** logging starts, so CUDA init and kernel autotuning aren't in the measurement
2. Wall-clock start and end timestamps of the workload are saved to a JSON next to the CSV, so the energy integral covers exactly the workload window
3. `min_new_tokens=50` and greedy decoding so every run generates exactly 50 tokens per sequence, which keeps runs comparable

Llama-3.2-1B, batch 1:

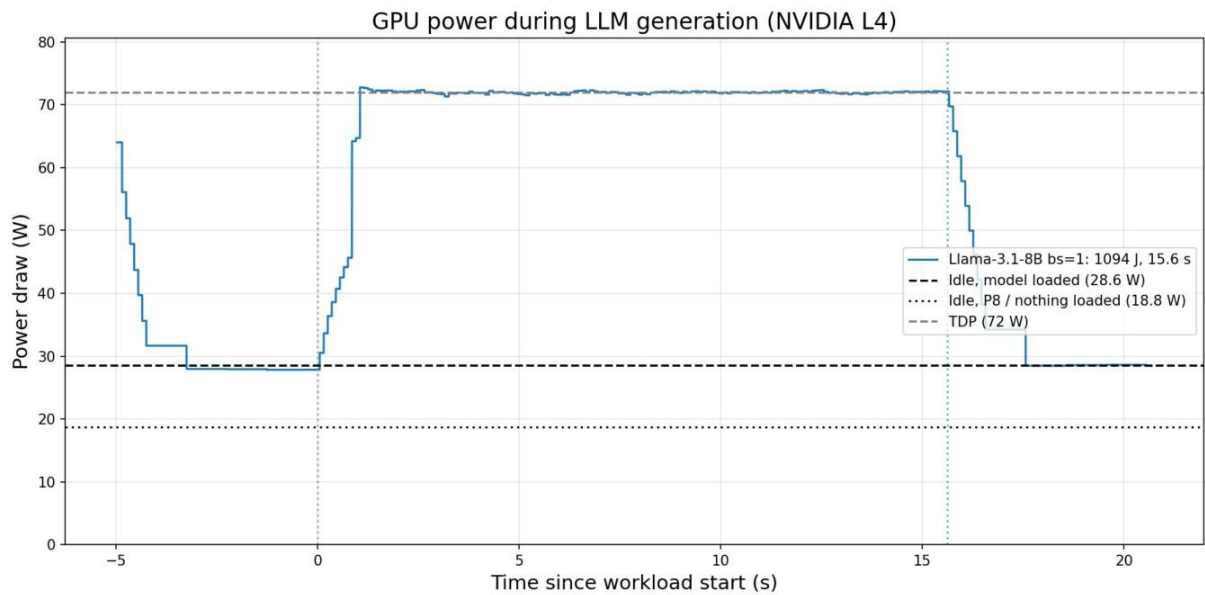


Time	Avg power	Peak	Energy	Per generate()	Per token
3.28 s	61.1 W	72.7 W	200 J	40.1 J	0.80 J

Shape of the trace: ~28 W before the run, a ~1 s ramp as clocks come up, then the card sits on the 72 W cap until the work ends, followed by a ~2 s decay back to ~28 W.

Two definitions of idle. The plain `nvidia-smi` in Part III showed 16 to 19 W. With a model loaded and the GPU recently active, it settles at ~28 W instead and doesn't drop to P8 within the 5 s padding. I plot both lines. For "energy above idle" I subtract the 28 W figure, since that's the real floor the workload sits on.

Exercise III: Power and Energy for Llama-3.1-8B



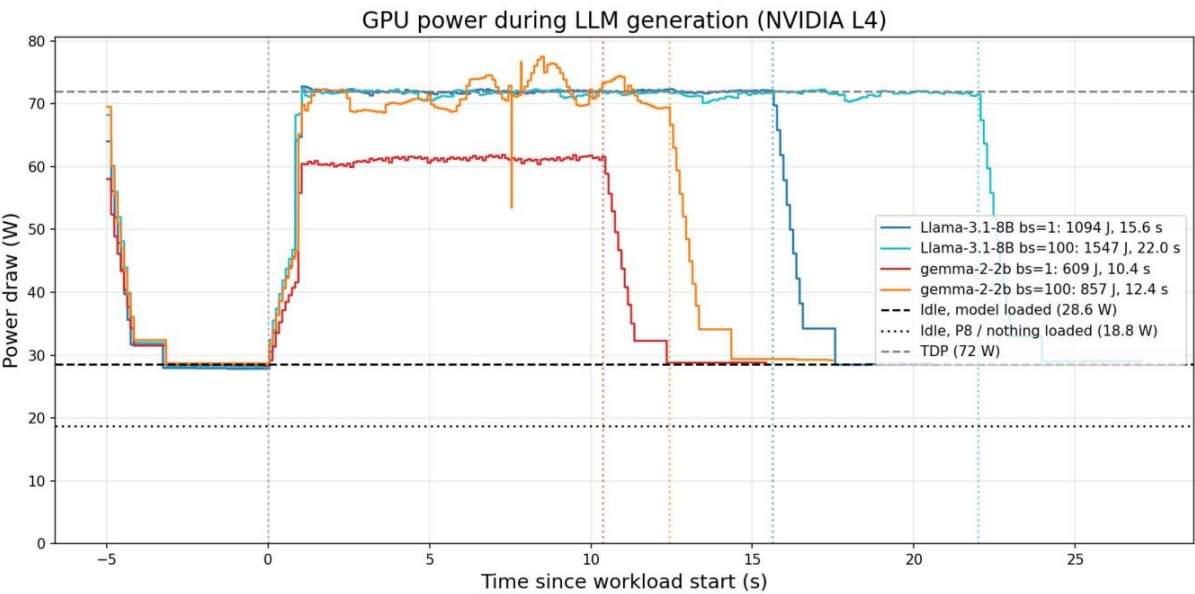
Energy is the integral of power over the workload window: $E = \sum P_i \cdot \Delta t_i$, i.e add up each power reading times the length of time it covered. Graphically, it's the area under the power curve between the start and end lines. That matches average power $\times$ time ($70.1\text{ W} \times 15.61\text{ s} = 1,094\text{ J}$).

Model	Time	Avg power	Idle	Energy	Energy above idle	Per generate()	Per token
Llama-3.1-8B, bs 1	15.61 s	70.1 W	28.6 W	1,094 J	648 J	218.8 J	4.38 J

Per forward pass: each `generate()` is 1 prefill pass plus 50 decode passes, so one forward pass costs about $218.8 / 51 \approx \mathbf{4.3\text{ J}}$.

What's going on: $5 \times 50 = 250$ tokens in 15.6 s is 62 ms per token. Each decode step reads all 8B fp16 weights ($\sim 16\text{ GB}$), so the card is streaming at about $16\text{ GB} / 62\text{ ms} \approx \mathbf{257\text{ GB/s}}$, or **86% of the L4's bandwidth**. This is GEMM B from Exercise II at the scale of a whole model: batch-1 decode is memory-bound, and it still draws the full 72 W because DRAM at full bandwidth is itself a big power consumer.

Exercise IV: Comparing Power Profiles Across Workloads



Model	Batch	Time	Avg power	Peak	Energy	Per token	Tokens/s
Llama-3.1-8B	1	15.61 s	70.1 W	72.8 W	1,094 J	4.38 J	16
Llama-3.1-8B	100	21.98 s	70.4 W	72.3 W	1,547 J	0.062 J	1,137
Gemma-2-2B	1	10.36 s	58.8 W	61.9 W	609 J	2.44 J	24
Gemma-2-2B	100	12.42 s	69.1 W	77.5 W	857 J	0.034 J	2,013

(Tokens/s = 5 iterations $\times$ 50 tokens $\times$ batch / time.)

```
Host: academ-acade-IL73aiWT6xf-c83014867702e61e.elb.us-east-1.amazonaws.com Themes: Default
e668a5cce97c9fd2e477e98c64ccb9b2 plot_power.py
Saved results/p4_llama1b.png

run time s avg W peak W idle W energy J J/iter J above idle
Llama-3.2-1B bs=1 3.28 61.1 72.7 28.0 200.4 40.1 108.6
energy per generated token: 0.802 J, median sample interval 1 ms
Saved results/ex3_llama8b_bs1.png

run time s avg W peak W idle W energy J J/iter J above idle
Llama-3.1-8B bs=1 15.61 70.1 72.8 28.6 1093.8 218.8 648.0
energy per generated token: 4.375 J, median sample interval 1 ms
Saved results/ex4_compare.png

run time s avg W peak W idle W energy J J/iter J above idle
Llama-3.1-8B bs=1 15.61 70.1 72.8 28.6 1093.8 218.8 648.0
energy per generated token: 4.375 J, median sample interval 1 ms
Llama-3.1-8B bs=100 21.98 70.4 72.3 29.0 1547.0 309.4 909.1
energy per generated token: 0.062 J, median sample interval 1 ms
gemma-2-2b bs=1 10.36 58.8 61.9 28.8 608.7 121.7 310.3
energy per generated token: 2.435 J, median sample interval 1 ms
gemma-2-2b bs=100 12.42 69.1 77.5 29.3 857.4 171.5 494.0
energy per generated token: 0.034 J, median sample interval 1 ms
(base) [nos673@ip-10-37-33-97 ml_workloads_part_3]$
```

Batch size barely changes power, it changes energy per token by ~70x. Going from batch 1 to 100 is 100x the tokens but only 1.4x the time for Llama and 1.2x for Gemma. Power stays near the cap either way, so total energy rises only 1.4x while energy per token drops 71x (Llama) and 72x (Gemma). The reason is the same as GEMM B: at batch 1 each weight read from DRAM is used for one token. At batch 100 the same read serves 100 tokens, so the extra work is nearly free until compute becomes the limit.

Gemma-2-2B at batch 1 doesn't reach the power cap. It sits at ~61 W while every other run pins 72 W. At 41 ms per token it is only moving ~5.2 GB of weights per step, about 127 GB/s, so it isn't bandwidth-bound either. My read is that it's overhead-bound: Gemma 2's architecture (logit softcapping, alternating sliding-window attention) launches more small kernels per token, and at batch 1 the GPU idles between them. I didn't profile it, so this is a hypothesis. A PyTorch Profiler trace of one decode step would confirm it.

Gemma at batch 100 briefly exceeds TDP (77.5 W peak). The power limiter works on an averaged window, so short spikes above 72 W are allowed.

Llama-8B vs Gemma-2B. Gemma is ~3x smaller, and per token it costs 44% less at batch 1 and 45% less at batch 100. That is far less than 3x, because at batch 1 Gemma isn't using the memory system efficiently, and at batch 100 both runs are pinned at the power cap, so the gap shows up as time rather than watts.

Takeaways

- The L4 is a low-bandwidth, power-capped part. Its ridge point is 403 FLOP/B, higher than an H100's, and any sustained workload pins it at 72 W.
- Decode is memory-bound and prefill is compute-bound. GEMM B ($M = 1$) hits 81% of DRAM bandwidth, and full-model decode for Llama-8B hits 86%.
- Batching is the biggest energy lever here: 100x batch cut energy per token by ~70x at nearly constant power.
- "Idle" needs a definition. 16 to 19 W in P8 with nothing loaded, ~28 W with a model resident right after work.

Glossary

Term	Meaning
AI (arithmetic intensity)	FLOPs per byte moved from memory. High AI = lots of reuse per byte
Bandwidth	Rate data moves between GPU memory and the math units (GB/s)
Batch size	Number of independent prompts processed together
Compute-bound	Speed is limited by the math units (right of the ridge)
Decode	Generating output tokens one at a time. Memory-bound at small batch
DRAM	The GPU's main memory (23 GB on the L4)
Energy (J)	Power × time. What you pay for
FLOP	One floating-point add or multiply
fp16	16-bit floating point numbers, 2 bytes each
GEMM	General matrix multiply, $C = A \times B$
Kernel	A single function launched on the GPU
Memory-bound	Speed is limited by memory bandwidth (left of the ridge)
nvidia-smi	NVIDIA's command-line GPU monitoring tool
P-state	GPU power mode. P0 = full speed, P8 = deep idle
Power (W)	Instantaneous rate of energy use
Prefill	Processing the whole prompt in one pass. Compute-bound
Ridge point	AI where a roofline turns flat. Peak FLOP/s / bandwidth
Roofline	Plot of attainable FLOP/s vs AI for a given chip
Slurm, srun, sbatch	The cluster's job scheduler and its interactive / batch job commands
Tensor Cores	Specialized GPU units for small matrix multiplies. The 121 TFLOP/s fp16 peak comes from these
TDP / power limit	Max power the GPU is allowed to draw. The L4 throttles its clocks to stay under 72 W
Token	A word or word-piece. LLMs generate one token per decode step